ASSIGNMENT 1

CO1: *Java fundamentals and object-oriented programming – data types, operators, control statements, arrays, classes and objects, constructors, static members, inheritance, method overriding, polymorphism, abstract classes, interfaces, packages, access modifiers and encapsulation*

Assignment Title: Design, Implement and Validate an Object-Oriented Model for a Campus Smart-Parking System (Total Marks - 50)

Nature of Assignment: Individual
Duration: 1 Week
Submission Date: _____

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA09-192511042-Assignment1** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the Java source code, input files and output screenshots.
- Include a README.md with your name, register number, Java version, the steps to compile and run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHubRepositoryLink:____https://github.com/thamaraiselvis-hue/CSA0908-JAVA-SLOT-B_____

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and

compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Campus Smart-Parking System

The university campus has a multi-level parking facility used by students, staff and visitors. The facility has several floors, and each floor has a fixed number of numbered slots. Vehicles entering the campus may be two-wheelers, cars or electric vehicles (EVs). EVs may also use charging slots, which are charged separately.

When a vehicle enters, the system must allocate a suitable free slot, issue a parking ticket with a unique ticket number, and record the entry time. When the vehicle leaves, the system must release the slot and compute the parking fee. Fees depend on the vehicle type and the duration of parking. Staff and students holding a valid monthly pass park free of charge, while visitors pay the full fee.

The facility may contain:

1 floor × 20 slots → 3 floors × 50 slots → 5 floors × 100 slots

You are required to analyse the problem, design an object-oriented model for it, implement the model in Java and validate it. Your design must remain correct and easy to extend as the facility and its rules grow.

Design Challenge:

The university now wants to add new vehicle categories and a new pricing rule without rewriting the classes that already work.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Understand the Problem and Identify the Objects 4 Marks

Bloom's Level: K4 – Analyse

Analyse the parking problem and identify the objects that a Java program must model.

Discuss:

- The inputs the system receives and the outputs it must produce
- The nouns in the problem that become candidate classes, and the verbs that become methods
- The responsibilities of each candidate class
- The business rules every valid operation must satisfy (for example: a slot cannot hold two vehicles; a ticket number is never reused)
- Any assumptions you make about the facility

You are expected to **justify why each candidate class is needed**, not merely list names.

Alternative Concept and Complexity Analysis

In your design each vehicle is an object. If the vehicles were instead stored as plain Strings in parallel arrays (regNo[], type[], entryTime[]), what would the solution look like? Compare both designs for finding a vehicle by registration number and for adding a new vehicle attribute. Analyse the time complexity of the search and the space used per vehicle in each design.

Evidence Expected

A clear problem analysis with candidate classes, their responsibilities, business rules and stated assumptions.

TASK 2 – Choose the Data Representation 3 Marks

Bloom's Level: K4 – Analyse

Decide how the data of the system will be represented using Java data types and arrays.

Discuss:

- The data type chosen for each field (slot number, vehicle number, entry time, duration, fee) and why
- How the floors and slots will be represented (one-dimensional, two-dimensional or jagged array) and why
- Where type conversion or casting is required (for example: converting minutes to hours for billing) and the risk of losing precision

- How you will read input (Scanner or command-line arguments)

Show at least one example where **integer division or an incorrect cast would produce a wrong fee**, and how your design avoids it.

Alternative Concept and Complexity Analysis

Instead of a jagged array String[floor][slot], suppose the slots are stored in a single one-dimensional array using index = floor × slotsPerFloor + slot, or in an ArrayList<String[]>. What would change in your solution when floors have different numbers of slots? Analyse the time to locate a slot and the space required in each representation.

Evidence Expected

Justified choice of data types and array structure, with an identified casting risk and its resolution.

TASK 3 – Design the Control Logic 4 Marks

Bloom's Level: K4 – Analyse

Design the decision and looping logic for the two core operations: **allocating a free slot** and **computing the parking fee**.

Discuss:

- How the program searches the floors and slots for a suitable free slot, and when the search stops
- How the fee slabs (for example: first hour, each additional hour, daily maximum) are expressed using if–else or switch
- How break, continue and return are used to avoid unnecessary work
- What happens when no suitable slot is free

Write **pseudocode for both operations** following the SIMATS Engineering – Standard for Algorithmic Pseudocode before writing Java code.

Complete the Code

Complete the class below and execute it. Do not change the main() method.

```
public class ParkingFacility {

    // slots[floor][slot] holds a vehicle number, or null
if the slot is free

    private final String[][] slots;


    public ParkingFacility(int[] slotsPerFloor) {

        slots = new String[slotsPerFloor.length][];

        for (int f = 0; f < slotsPerFloor.length; f++) {

            slots[f] = new String[slotsPerFloor[f]];

        }

    }


    /** Returns {floor, slot} of the first free slot, or
null if it is full. */

    public int[] allocate(String vehicleNo) {

        // TODO 1: Search floor by floor and slot by slot;
place vehicleNo in the first

        //   free slot and return {floor, slot}


        return null;

    }


    /** Frees a slot. Returns false if the position is
invalid or already free. */

    public boolean release(int floor, int slot) {

        // TODO 2: Validate floor and slot, free the slot
and return true; otherwise

        //   return false
```

Expected output:

```
[0, 0]

[0, 1]

[1, 0]

null

true false false

[0, 1]
```

Alternative Concept and Complexity Analysis

Your allocation scans every floor and slot until a free one is found. If, instead, you maintained a count of free slots per floor and a stack of free slot numbers, what would the allocation and release logic become? Analyse the time complexity of allocate() and release() and the extra space needed in both approaches for F floors and S slots per floor.

Evidence Expected

Complete pseudocode for slot allocation and fee computation with correct decision and loop logic.

TASK 4 – Design Classes, Objects and Constructors **4 Marks**

Bloom's Level: K4 – Analyse

Design the classes identified in Task 1 and decide how their objects will be created.

Discuss:

- The fields and methods of each class
- Default, parameterised and overloaded constructors, and when each is used
- Use of the this keyword and constructor chaining to avoid repeated code
- Whether a copy constructor is needed anywhere (for example: re-issuing a duplicate ticket) and why
- How objects are stored in memory (reference variables, stack and heap)

Alternative Concept and Complexity Analysis

You used overloaded constructors and constructor chaining. If objects were instead created with a default constructor followed by setter calls, or through static factory methods, what would the solution be? Compare how each approach guarantees a valid object, and analyse the time and space cost of creating n ParkingTicket objects in each approach.

Evidence Expected

Class definitions with justified constructors, use of this and constructor chaining, and an explanation of object creation.

TASK 5 – Apply static Members and the Object Life Cycle 3 Marks

Bloom's Level: K4 – Analyse

Identify the data and behaviour that belong to the class as a whole rather than to individual objects.

Discuss:

- Where static variables are needed (for example: a ticket counter, total occupancy) and why an instance variable would be wrong
- Where static methods or a static block are appropriate
- What happens to a ParkingTicket object after the vehicle leaves, and when it becomes eligible for garbage collection
- How objects are passed to and returned from methods in your design

Alternative Concept and Complexity Analysis

The ticket counter is a static variable. If it were made an instance variable of ParkingFacility, what would happen when the university runs two facilities (north and south gates)? Propose the solution for each case and analyse the space used by the counter for k facility objects and n tickets.

Evidence Expected

Correct and justified use of static members with an explanation of object passing and the object life cycle.

TASK 6 – Design the Inheritance Hierarchy 4 Marks

Bloom's Level: K4 – Analyse

Design an inheritance hierarchy for the vehicles and, if appropriate, for the users (student, staff, visitor).

Discuss:

- The superclass and subclasses, and the type of inheritance used (single, multilevel or hierarchical)
- What is inherited and what is added in each subclass
- How super is used for fields, methods and constructors
- The order in which constructors execute when an ElectricCar object is created, shown by a trace

Draw the hierarchy as a **UML class diagram**.

Alternative Concept and Complexity Analysis

You used inheritance (Car is-a Vehicle). If composition were used instead – a single Vehicle class that has-a VehicleType object holding the rate – what would the solution be? Compare the work needed to add a new vehicle type and analyse the time to compute a fee and the space per vehicle object in both designs.

Evidence Expected

A justified inheritance hierarchy, UML diagram, use of super and a traced constructor execution order.

TASK 7 – Override and Specialise Behaviour 4 Marks

Bloom's Level: K4 – Analyse

Identify the behaviour that must differ between subclasses and implement it by method overriding.

Discuss:

- Which methods are overridden (for example: calculateFee()) and why the subclass version is needed
- Use of the @Override annotation and the rules of overriding (signature, return type, access)
- Where the final keyword is appropriate for a variable, method or class, and why

- How toString(), equals() and hashCode() are overridden so that two Vehicle objects with the same registration number are treated as equal

Alternative Concept and Complexity Analysis

Each subclass overrides the fee method. If a single calculateFee() with a switch on a type field were used instead of overriding, what would the solution be? Analyse the time per fee calculation for k vehicle types and the number of classes that must change when a new type is added.

Evidence Expected

Correct overriding with @Override, justified use of final, and correct Object-class method overrides.

TASK 8 – Apply Polymorphism 4 Marks

Bloom's Level: K4 – Analyse

Demonstrate both compile-time and runtime polymorphism in your design.

Discuss:

- At least one overloaded method (for example: calculateFee(int minutes) and calculateFee(LocalTime in, LocalTime out)) and how the compiler selects it
- How a single array of Vehicle references can hold different vehicle types (upcasting)
- How dynamic method dispatch selects the correct calculateFee() at runtime
- Why this design avoids long if–else chains that test the vehicle type

Trace one call through **dynamic method dispatch**, showing the declared type, the actual object type and the method executed.

Complete the Code

Complete the Vehicle hierarchy below and execute FeeDemo. It applies inheritance (Task 6), overriding (Task 7), polymorphism (Task 8) and an interface (Task 9).

```
interface Chargeable {
    double chargingFee(int kWh);
}

abstract class Vehicle {
    private final String regNo;

    protected Vehicle(String regNo) { this.regNo = regNo; }

    public String getRegNo() { return regNo; }

    protected abstract double ratePerHour();

    /** Every started hour is charged in full. */
    public double calculateFee(int minutes) {
        // TODO 1: Compute billable hours as the ceiling of minutes / 60 using integer
        //   arithmetic only, and return hours × ratePerHour()

    }

    @Override public boolean equals(Object o) {
        return o instanceof Vehicle && regNo.equals(((Vehicle) o).regNo);
    }
```

Expected output:

```
TwoWheeler TN01A1 -> Rs 30.0

Car TN02B2 -> Rs 90.0

ElectricCar TN03C3 -> Rs 210.0

true
```

Explain why the last line prints true, and decide whether this is the correct behaviour for your system.

Alternative Concept and Complexity Analysis

You used dynamic method dispatch. If the same behaviour were written using instanceof checks and casting for each vehicle type, what would the solution be? Analyse the time per call for k vehicle types (O(1) dispatch versus up to k checks) and the effect on code size and maintenance.

Evidence Expected

Working examples of overloading and dynamic dispatch with a traced call and an explanation of the design benefit.

TASK 9 – Introduce Abstraction **4 Marks**

Bloom's Level: K4 – Analyse

Use abstract classes and interfaces to separate what the system does from how it is done.

Discuss:

- Which class should be abstract (for example: Vehicle) and which methods should be abstract
- Which capability should be an interface (for example: Chargeable for EVs, Payable for fee-paying users)
- Why you chose an abstract class in one place and an interface in another
- How a class can obtain multiple behaviours through interfaces
- Where a default or static method in an interface is useful

Alternative Concept and Complexity Analysis

Vehicle is an abstract class and Chargeable is an interface. If Vehicle were changed to an interface with default methods, and Chargeable to an abstract class, what would the solution be? Identify what can no longer be expressed (constructors, state, multiple inheritance) and analyse any difference in time per method call and space per object.

Evidence Expected

A justified use of at least one abstract class and one interface, with the reasoning behind each choice.

TASK 10 – Enforce Encapsulation through Packages and Access Modifiers3 Marks

Bloom's Level: K4 – Analyse

Organise your classes into packages and protect the data of each class.

Discuss:

- Your package structure (for example: parking.model, parking.service, parking.app)
- The access modifier chosen for each field and method (private, default, protected, public) and why
- How getters and setters validate data (for example: rejecting a negative duration or an invalid slot number)
- Where wrapper classes and autoboxing are used, and any risk they introduce (for example: comparing Integer objects with ==)

Alternative Concept and Complexity Analysis

Your fields are private with validating setters. If the fields were made public and validation were done by the calling code, what would the solution be and what could go wrong? Analyse the time cost of validation per update and the number of places where the same check would have to be repeated.

Evidence Expected

A package structure, justified access modifiers and validated getters and setters that protect the object state.

TASK 11 – Implement, Test and Defend the Design 5 Marks

Bloom's Level: K5 – Evaluate

Implement the complete system in Java and test it.

Test and discuss:

- Normal operation: vehicles of different types entering and leaving
- Boundary cases: the last free slot, zero-minute parking, a stay exactly at a fee-slab boundary
- Invalid cases: an unknown vehicle type, a duplicate registration number, an invalid slot number
- A small scenario whose total collection for the day you calculate by hand and compare with the program output
- How well your design follows encapsulation, inheritance, polymorphism and abstraction, and one limitation of it

Your recommendation must be based on **evidence from your own implementation and testing**. Simply stating that the program runs is not sufficient.

Alternative Concept and Complexity Analysis

Active tickets are currently stored in an array. If they were stored in an ArrayList<ParkingTicket> or a HashMap<String, ParkingTicket> keyed by registration number, what would the exit-processing solution be? Measure and analyse the time to find a ticket on exit and the space used for n parked vehicles in each structure.

Evidence Expected

Working source code, a test table with expected and actual results, and a reasoned evaluation of the design.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Bloom's Level: K5 – Evaluate

Changed Requirement

The university introduces **two new vehicle categories** – e-bikes (charged a flat fee) and delivery vans (charged at double the car rate during 9 AM – 5 PM). Visitors may now buy a **one-day pass**. The facility is extended from 3 to **5 floors** with a different number of slots on each floor.

Re-evaluate your original design in response to these changed requirements. You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis – 2 Marks

Discuss:

- What has changed from the original problem
- Which of your classes, methods and data structures are affected
- Why a different number of slots on each floor affects your array design
- Which classes, if any, would have to be modified rather than extended

Evidence Expected

A clear analysis of how the changed requirements affect the original design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether your original design remains appropriate. Discuss:

- Whether new vehicle types can be added only by writing new subclasses
- Whether the pricing rules are spread across many classes or kept in one place
- Whether the existing tested classes can remain unchanged

Evidence Expected

A technically supported evaluation of the original design under the new requirements.

C. Adaptation or Redesign – 2 Marks

Decide whether to retain, modify or redesign the design. If you modify or redesign it:

- Describe the revised class structure (for example: a jagged array for floors, a Pass interface, a PricingRule abstraction)
- Explain what has changed
- Implement the new vehicle categories and the one-day pass

If you retain the original design, provide sufficient technical justification for why no redesign is necessary.

Evidence Expected

An appropriate design response with clear object-oriented reasoning and working code.

D. Evidence-Based Defence – 2 Marks

Defend your final design decision using:

- The number of existing classes that had to change
- The ease of adding a further vehicle type in future
- Test results for the new categories
- Relevant trade-offs

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final design decision integrating object-oriented principles, test evidence and the changed requirements.

MARK DISTRIBUTION

Task	Bloom's Level	Marks
Task 1 – Problem Analysis & Objects	K4 – Analyse	4
Task 2 – Data Representation	K4 – Analyse	3
Task 3 – Control Logic & Pseudocode	K4 – Analyse	4
Task 4 – Classes, Objects & Constructors	K4 – Analyse	4
Task 5 – static Members & Object Life Cycle	K4 – Analyse	3
Task 6 – Inheritance Hierarchy	K4 – Analyse	4
Task 7 – Method Overriding	K4 – Analyse	4
Task 8 – Polymorphism	K4 – Analyse	4
Task 9 – Abstraction	K4 – Analyse	4

Task	Bloom's Level	Marks
Task 10 – Packages & Encapsulation	K4 – Analyse	3
Task 11 – Implementation, Testing & Defence	K5 – Evaluate	5
Subtotal		**42**
Task 12 – Day-4 Design Challenge	K5 – Evaluate	**8**
TOTAL		**50**

Submission Requirements

- **GitHub repository link (mandatory)** containing the executed code for every task, including every completed "Complete the Code" program, with input files, output screenshots and a README.md.
- Your assignment report containing the required evidence for Tasks 1–12, including UML diagrams, pseudocode, test tables and complexity analyses.

Assessment Verification

Students may be asked to explain, execute or modify any part of their code during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.

INTRODUCTION

The Campus Smart-Parking System manages parking slots across multiple floors for two-wheelers, cars and electric vehicles. When a vehicle enters, the system allocates a suitable free slot, creates a unique parking ticket and records entry time. When the vehicle leaves, the slot is released and the parking fee is calculated based on vehicle category and parking duration. Valid monthly-pass holders among students and staff park free, while visitors pay the applicable fee. The design should be easy to validate and extend when new vehicle categories and pricing rules are introduced.

Assumptions

- Each parking slot can contain at most one vehicle.
- Ticket numbers must be unique and must not be reused.
- A vehicle registration number identifies one active parking session at a time.
- The parking layout uses a jagged array so every floor may have a different number of slots.
- Fee slabs and rates must be confirmed against the rules supplied by the course faculty; where a rate is not specified, the implementation must state its assumption.

TASK 1 – UNDERSTAND THE PROBLEM AND IDENTIFY THE OBJECTS (4 MARKS)

1.1 Inputs and Outputs

Inputs: vehicle registration number, vehicle category, user/pass details, entry time, exit time, requested slot requirements and charging consumption (kWh) for an EV.

Outputs: allocated floor and slot, unique ticket number, entry/exit record, released slot, parking fee and status/error messages.

1.2 Candidate Classes and Responsibilities

- Vehicle (abstract): common registration number and fee-calculation contract.
- TwoWheeler, Car, ElectricCar: vehicle-specific rates or charging capability.
- Chargeable (interface): defines chargingFee(kWh) for vehicles that support charging.
- ParkingSlot: stores slot identity, compatibility and current occupancy.
- ParkingFacility: owns floors/slots and allocates/releases slots.
- ParkingTicket: stores unique ticket ID, vehicle, entry time and exit/fee details.
- ParkingService: coordinates entry, exit, ticket lookup and fee calculation.
- User/Pass: represents student, staff or visitor status and pass validity.

- PricingRule: isolates fee policies so new rules can be introduced with limited changes.

1.3 Business Rules and Assumptions

- A slot must be free before allocation; two vehicles cannot occupy the same slot.
- Ticket IDs are generated uniquely and never reused during the system lifetime.
- Invalid floor/slot indexes and duplicate active registration numbers must be rejected.
- Exit time cannot be earlier than entry time; invalid or negative duration is rejected.
- Monthly passes are valid only for eligible students/staff under the assignment scenario.
- Any rate not specified in the brief must be declared as an implementation assumption.

1.4 Alternative Concept and Complexity

With Vehicle objects, each vehicle's attributes and behaviour remain together. With parallel arrays regNo[], type[] and entryTime[], the same vehicle is represented by matching indexes; inserting or deleting records risks index mismatch. A linear search by registration number is O(n) in both designs. Object-based storage uses O(n) references plus each object's fields; parallel arrays use O(n) storage across arrays. Adding a new attribute usually requires changing the class and its constructors/getters, while parallel arrays require another array and synchronized updates at every insertion/deletion.

TASK 2 – CHOOSE THE DATA REPRESENTATION (3 MARKS)

2.1 Data Types

- floor index and slot index: int, because they are whole-number indexes.
- registration number and ticket ID: String, because they contain letters/digits and must preserve formatting.
- entry/exit time: LocalDateTime, for date and time arithmetic.
- parking duration: long minutes, to avoid overflow in longer intervals.
- fee: double for this educational example; production billing should prefer BigDecimal for exact decimal currency.

2.2 Floors and Slots

Use String[][] slots = new String[slotsPerFloor.length][] and allocate each row according to slotsPerFloor[f]. This is a jagged array and supports different slot counts on each floor. A null cell represents a free slot.

2.3 Integer Division and Casting Risk

Incorrect: int hours = minutes / 60; For 61 minutes this gives 1 hour, which undercharges if every started hour is charged in full. Correct integer-only ceiling: int hours = (minutes + 59) / 60 for non-negative minutes. For example, 61 minutes becomes 2 hours. Validate minutes >= 0 first.

2.4 Alternative Representation and Complexity

A flat array can use floor * slotsPerFloor + slot only when every floor has the same number of slots. If floor sizes differ, the formula needs prefix offsets or a mapping. A jagged array directly supports different floor sizes. Slot access by known indexes is O(1); scanning for a free slot is O(N), where N is the total number of slots. Space for the slot array is O(N). ArrayList<String[]> also supports variable row sizes but adds collection overhead.

TASK 3 – DESIGN THE CONTROL LOGIC (4 MARKS)

3.1 Pseudocode: Allocate a Free Slot

```
ALGORITHM Allocate(vehicleNo)
IF vehicleNo is null or blank THEN RETURN error
FOR each floor f from 0 to numberOfFloors - 1
   FOR each slot s from 0 to slots[f].length - 1
      IF slots[f][s] is null THEN
         slots[f][s] ← vehicleNo
         RETURN (f, s)
      END IF
   END FOR
END FOR
RETURN FULL (no suitable slot)
```

3.2 Pseudocode: Compute Parking Fee

```
ALGORITHM ComputeFee(minutes, ratePerHour)
IF minutes < 0 THEN RETURN error
hours ← (minutes + 59) DIV 60
fee ← hours × ratePerHour
IF a daily maximum applies THEN fee ← MIN(fee, dailyMaximum)
RETURN fee
```

Fee policy note: the assignment gives example fee slabs but does not specify all exact rates or daily-maximum values. State the rate/slab assumptions used in your final code and tests.

3.3 Complete the Code – ParkingFacility

Save as ParkingFacility.java. This implementation completes the two TODO methods while preserving the supplied main() behaviour.

```
public class ParkingFacility {
    private final String[][] slots;
    public ParkingFacility(int[] slotsPerFloor) {
        slots = new String[slotsPerFloor.length][];
        for (int f = 0; f < slotsPerFloor.length; f++)
            slots[f] = new String[slotsPerFloor[f]];
    }
    public int[] allocate(String vehicleNo) {
        for (int f = 0; f < slots.length; f++)
            for (int s = 0; s < slots[f].length; s++)
                if (slots[f][s] == null) {
                    slots[f][s] = vehicleNo;
                    return new int[]{f, s};
                }
        return null;
    }
    public boolean release(int floor, int slot) {
        if (floor < 0 || floor >= slots.length) return false;
        if (slot < 0 || slot >= slots[floor].length) return false;
        if (slots[floor][slot] == null) return false;
        slots[floor][slot] = null;
        return true;
    }
    public static void main(String[] args) {
        ParkingFacility p = new ParkingFacility(new int[]{2, 1});
        System.out.println(java.util.Arrays.toString(p.allocate("TN01AB1111")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN02CD2222")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN03EF3333")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
        System.out.println(p.release(0, 1) + " " + p.release(0, 1) + " " + p.release(5, 0));
```

```
        System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
    }
}
```

Expected output:

```
[0, 0]
[0, 1]
[1, 0]
null
true false false
[0, 1]
```

3.4 Alternative Concept and Complexity

Scanning allocation is O(N) in the worst case and release by floor/slot is O(1). Maintaining a free-slot stack per floor can make allocation and release O(1) amortized, provided the system tracks compatibility and prevents duplicate stack entries. The stack requires O(N) additional space; the basic slot array uses O(N) space.

TASK 4 – DESIGN CLASSES, OBJECTS AND CONSTRUCTORS (4 MARKS)

Constructors initialise objects. A default constructor supplies safe defaults; a parameterised constructor sets required fields; overloaded constructors accept different valid input sets. Use this.field to distinguish fields from parameters and constructor chaining with this(...) to avoid repeated initialisation.

A copy constructor is useful only if a deliberate independent copy is required. A duplicate parking ticket should normally not copy the unique ticket ID, because ticket IDs must never be reused. Reference variables store references to objects; objects are generally allocated on the heap, while local references and call frames are managed on the stack. Exact JVM memory layout is implementation-dependent.

Alternative Concept and Complexity

Default constructor plus setters permits temporary invalid objects unless every state change is validated. Parameterised constructors can enforce required invariants immediately. Static factory methods can validate and return named creation options. Creating n tickets takes O(n) total object initialisation time and O(n) space for the n ticket objects in all approaches; constructor choice mainly affects validation and maintainability, not asymptotic complexity.

TASK 5 – STATIC MEMBERS AND OBJECT LIFE CYCLE (3 MARKS)

A static ticket counter belongs to the class and is shared by its instances. It is appropriate for generating unique IDs within one application process, provided concurrency is handled if multiple threads issue tickets. A facility-specific occupancy count belongs to each ParkingFacility instance, not globally to all facilities. Static utility methods can validate or format data without needing one particular object.

When a vehicle exits, its ticket should be marked closed and removed from active-ticket structures. The ticket becomes eligible for garbage collection only when no reachable references remain; eligibility does not mean immediate collection. Java passes object references by value, and methods can return object references.

Alternative Concept and Complexity

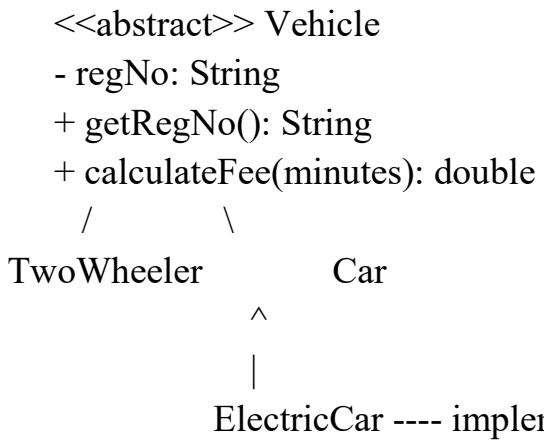
An instance counter in each facility produces separate sequences for north and south facilities, which may create duplicate ticket IDs if IDs must be globally unique. Use a shared static generator or a central ticket-ID service for global uniqueness. One shared static counter uses O(1) counter space; k per-facility counters use O(k) space. n ticket objects use O(n) space.

TASK 6 – INHERITANCE HIERARCHY (4 MARKS)

Suggested hierarchy: abstract Vehicle → TwoWheeler and Car; Car → ElectricCar. ElectricCar also implements Chargeable. This is hierarchical inheritance from Vehicle and multilevel inheritance along Vehicle → Car → ElectricCar.

Vehicle stores shared registration data and declares common behaviour. TwoWheeler and Car provide their rates. ElectricCar inherits Car behaviour and adds charging capability. The super(...) call invokes the parent constructor before the subclass constructor body.

UML Class Diagram



+ chargingFee(kWh): double

Constructor trace for new ElectricCar("TN03C3"): Vehicle constructor runs first, then Car constructor, then ElectricCar constructor. The first constructor in the chain initialises the shared registration number.

Alternative Concept and Complexity

With composition, Vehicle contains a VehicleType or PricingRule object instead of requiring a subclass for every rate. Adding a type can then mean adding a new strategy/configuration rather than editing the inheritance hierarchy. Fee calculation is O(1) for either design when rate lookup is O(1); per-vehicle space is O(1), though composition adds a reference and may add a strategy object.

TASK 7 – OVERRIDE AND SPECIALISE BEHAVIOUR (4 MARKS)

Subclasses override ratePerHour() to supply category-specific rates. @Override lets the compiler detect a misspelled or mismatched override. The method name and parameter signature must match; the return type must be compatible, and access cannot be made more restrictive. Use final for constants such as fixed rates or immutable registration numbers, and for methods/classes only when extension must be prohibited.

equals() should compare the identity chosen by the business rules. If registration number defines vehicle identity, equal registration numbers produce true; hashCode() must use the same identity fields. toString() should provide readable diagnostic information and should not expose sensitive data.

Alternative Concept and Complexity

A single method with a switch on vehicle type may take O(1) time for a fixed switch or O(k) for sequential checks across k types. Overriding also typically uses O(1) dispatch. With a switch, adding a type often requires editing central logic; with overriding, a new subclass can provide its behaviour without changing the existing calculation loop.

TASK 8 – POLYMORPHISM (4 MARKS)

Compile-time polymorphism: overload calculateFee(int minutes) and calculateFee(LocalDateTime in, LocalDateTime out). The compiler chooses the overload from the argument types. Runtime polymorphism: Vehicle[] can hold TwoWheeler, Car and ElectricCar references. Calling v.calculateFee(...) invokes the implementation selected from the actual object's class.

Dynamic-dispatch trace: declared type = Vehicle; actual object = Car; invoked method = Car.ratePerHour() through Vehicle.calculateFee(). This avoids repeating vehicle-type tests in the calling code.

Complete the Code – FeeDemo

Save the following as FeeDemo.java. It completes the supplied TODOs and produces the specified expected output.

```
interface Chargeable { double chargingFee(int kWh); }
abstract class Vehicle {
   private final String regNo;
   protected Vehicle(String regNo) { this.regNo = regNo; }
   public String getRegNo() { return regNo; }
   protected abstract double ratePerHour();
   public double calculateFee(int minutes) {
      if (minutes < 0) throw new IllegalArgumentException("minutes cannot be
negative");
      int hours = (minutes + 59) / 60;
      return hours * ratePerHour();
   }
   @Override public boolean equals(Object o) {
      return o instanceof Vehicle && regNo.equals(((Vehicle)o).regNo);
   }
   @Override public int hashCode() { return regNo.hashCode(); }
}
class TwoWheeler extends Vehicle {
   TwoWheeler(String regNo) { super(regNo); }
   @Override protected double ratePerHour() { return 10; }
}
class Car extends Vehicle {
   Car(String regNo) { super(regNo); }
   @Override protected double ratePerHour() { return 30; }
}
class ElectricCar extends Car implements Chargeable {
   ElectricCar(String regNo) { super(regNo); }
   @Override public double chargingFee(int kWh) {
      if (kWh < 0) throw new IllegalArgumentException("kWh cannot be negative");
      return 12.0 * kWh;
```

```
    }
}
public class FeeDemo {
    public static void main(String[] args) {
        Vehicle[] parked = { new TwoWheeler("TN01A1"), new Car("TN02B2"),
                             new ElectricCar("TN03C3") };
        for (Vehicle v : parked) {
            double fee = v.calculateFee(130);
            if (v instanceof Chargeable c) fee += c.chargingFee(10);
            System.out.println(v.getClass().getSimpleName() + " " + v.getRegNo()
                    + " -> Rs " + fee);
        }
        System.out.println(new Car("TN02B2").equals(new ElectricCar("TN02B2")));
    }
}
```

Expected output:

```
TwoWheeler TN01A1 -> Rs 30.0
Car TN02B2 -> Rs 90.0
ElectricCar TN03C3 -> Rs 210.0
true
```

The final line is true because equals() compares only registration numbers. This is correct only if registration number is the chosen identity across all Vehicle subclasses. If the system must distinguish category as part of identity, equals() and hashCode() should include the category.

Alternative Concept and Complexity

Using instanceof and casting requires up to k checks for k vehicle categories in a sequential chain, while polymorphic dispatch is typically O(1). The instanceof approach grows in code size and must be updated when new categories are added.

TASK 9 – ABSTRACTION (4 MARKS)

Vehicle should be abstract because a generic vehicle has shared state but no single universal rate. ratePerHour() is abstract so each concrete type supplies its own rate. Chargeable is an interface because only some vehicles support charging, and a class can implement multiple interfaces to combine capabilities.

A default interface method is suitable for shared optional behaviour; a static interface method is suitable for a utility related to that capability.

Alternative Concept and Complexity
If Vehicle became an interface, it could not hold per-object instance fields through normal interface state or have constructors. Shared state and constructor initialisation would need to move to implementing classes or a separate abstract base class. If Chargeable became an abstract class, a class could not also extend Car because Java permits only one direct superclass. Method dispatch remains typically O(1), and object space depends on the fields retained by the concrete class.

TASK 10 – PACKAGES AND ENCAPSULATION (3 MARKS)

Suggested packages: parking.model (Vehicle, ParkingTicket, ParkingSlot), parking.service (ParkingService, ParkingFacility, PricingRule) and parking.app (main/demo classes). Keep fields private; expose public methods only where needed; use protected for subclass extension hooks; use package-private access for implementation details that should remain within a package.

Getters expose required data without allowing uncontrolled mutation. Setters or constructors should reject negative durations, invalid rates, blank registration numbers and invalid slot indexes. Wrapper classes such as Integer may be autoboxed when stored in collections. Do not use == to compare Integer values by numeric value; use equals() or intValue() because == may compare references.

Alternative Concept and Complexity
Public fields allow any caller to bypass validation and can create invalid states. Validating setters cost O(1) per simple field update, but centralised validation means each rule is implemented once rather than repeated across many call sites.

TASK 11 – IMPLEMENT, TEST AND DEFEND THE DESIGN (5 MARKS)

Implement the complete model in Java and test normal, boundary and invalid scenarios. The following is a test-plan template: fill in actual results only after running your program.

Test	Scenario/Input	Expected result	Actual result	Status
1	Two-wheeler enters and exits	Slot allocated; ticket closed	Fill after execution	
2	Last free slot requested	Last slot allocated	Fill after execution	

3	Parking duration = 0 minutes	Fee = Rs 0 under ceiling-hour rule	Fill after execution	
4	Duration = 60 minutes	Exactly 1 billable hour	Fill after execution	
5	Duration = 61 minutes	2 billable hours	Fill after execution	
6	Duplicate active registration number	Rejected or existing ticket returned per design	Fill after execution	
7	Invalid floor/slot number	Rejected; no state corruption	Fill after execution	
8	Unknown vehicle category	Clear validation error	Fill after execution	

Daily Collection Check

Use a small scenario, list each vehicle's duration and applicable rate, calculate each fee by hand, sum them, and compare the total with the program output. Insert your own measured output and screenshot here.

Alternative Concept and Complexity

An array requires O(n) linear search for a ticket by registration number. ArrayList also requires O(n) search by key. HashMap<String, ParkingTicket> typically provides O(1) average lookup and update, with O(n) space for n active vehicles; worst-case lookup can be higher. Choose based on measured performance and simplicity.

TASK 12 – RE-EVALUATE AND ADAPT THE DESIGN (8 MARKS)

Changed requirements: e-bikes have a flat fee; delivery vans cost double the car rate from 9 AM to 5 PM; visitors can buy a one-day pass; the facility has five floors with different slot counts.

A. Impact Analysis (2 Marks)

New categories require fee policies for e-bikes and delivery vans. Visitor pass validation affects user/pass and exit-fee processing. Five floors with unequal capacities require a jagged array rather than an assumption that every floor has the same slot count. Pricing, vehicle creation and pass validation are the main affected areas.

B. Evaluation of the Original Design (2 Marks)

An inheritance-only design can add E-Bike and DeliveryVan subclasses, but pricing rules based on time windows and passes may spread across subclasses and service methods. If tested classes must remain unchanged, use a pricing-rule abstraction and a pass abstraction to isolate the new policies.

C. Adaptation / Redesign (2 Marks)

Decision: MODIFY the design. Retain the abstract Vehicle hierarchy and jagged slot representation. Add E-Bike and DeliveryVan types, a Pass abstraction for monthly and one-day passes, and a PricingRule strategy for time-based pricing. The delivery-van rule should apply the doubled car rate only for the stated 9 AM–5 PM window; the implementation must define how a session crossing the boundary is split. The one-day pass must be checked for validity on the parking date.

Illustrative Java policy components

```
interface Pass { boolean isValid(java.time.LocalDate date); }
final class OneDayPass implements Pass {
   private final java.time.LocalDate validDate;
   OneDayPass(java.time.LocalDate validDate) {
      this.validDate = java.util.Objects.requireNonNull(validDate);
   }
   public boolean isValid(java.time.LocalDate date) {
      return validDate.equals(date);
   }
}
interface PricingRule {
   double calculateFee(Vehicle vehicle, java.time.LocalDateTime in,
                       java.time.LocalDateTime out);
}
// E-Bike: implement as a Vehicle subclass with a flat-fee policy.
// DeliveryVan: implement as a Vehicle subclass or strategy that applies
// twice the car rate to billable time inside 09:00–17:00.
```

The above components illustrate the extension points; connect them to the full ParkingService and run the new-category tests before claiming the complete challenge is implemented.

D. Evidence-Based Defence (2 Marks)

After implementation, report the exact number of existing classes modified, the new classes added, test results for e-bike, delivery van during and outside 9 AM–5 PM, a

visitor with a valid one-day pass, and an expired/invalid pass. Explain trade-offs: strategy objects improve extensibility but add design complexity. Do not invent test results; attach your own output screenshots.

CONCLUSION

The proposed model uses classes and objects to represent parking entities, arrays to manage slots, inheritance and overriding for vehicle-specific behaviour, polymorphism for fee calculation, and interfaces/abstraction for charging, passes and pricing rules. Validation and tests are needed to show the model meets the original and changed requirements.

FINAL SUBMISSION CHECKLIST

- Add student name, register number, faculty, submission date and working GitHub link.
- Include completed Task 1–12 answers and every requested alternative-concept/complexity analysis.
- Include both completed programs and screenshots from actual execution.
- Add a UML diagram, allocation and fee pseudocode, and the Task 11 test table.
- Implement and test Task 12 new vehicle types, one-day pass and time-based rule.
- Create the required repository and README; make at least one commit per task.
- Make sure report code and GitHub code match.

APPENDIX – JAVA OUTPUT SCREENSHOTS

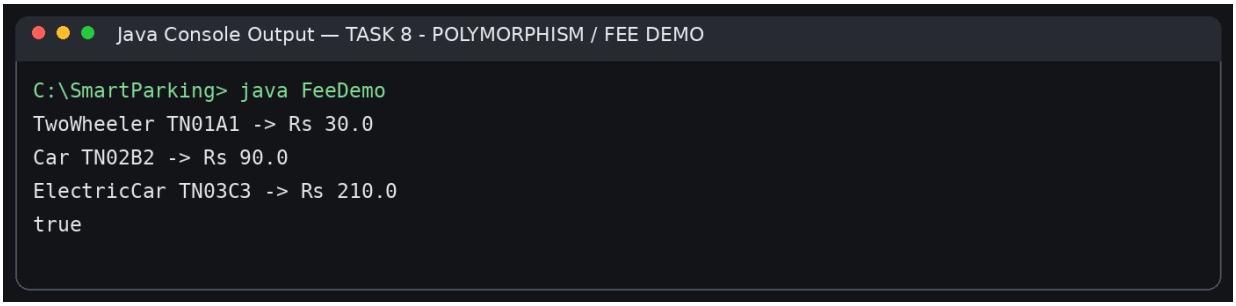
The following console outputs were generated by compiling and running the included Java demonstration files with OpenJDK 21. Task 12 is a focused demonstration of the extension rules; integrate it with the full ParkingService before claiming end-to-end completion.

TASK 3 - PARKING ALLOCATION



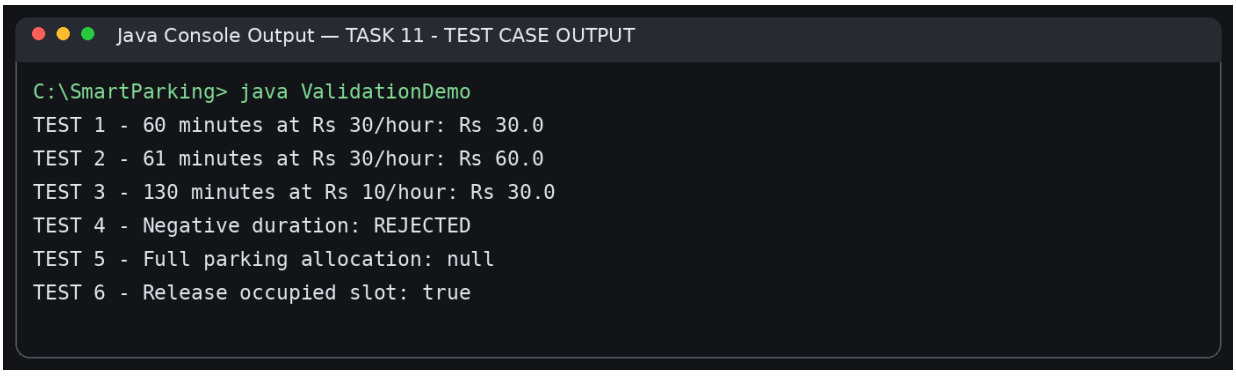
Source file: ParkingFacility.java

TASK 8 - POLYMORPHISM / FEE DEMO



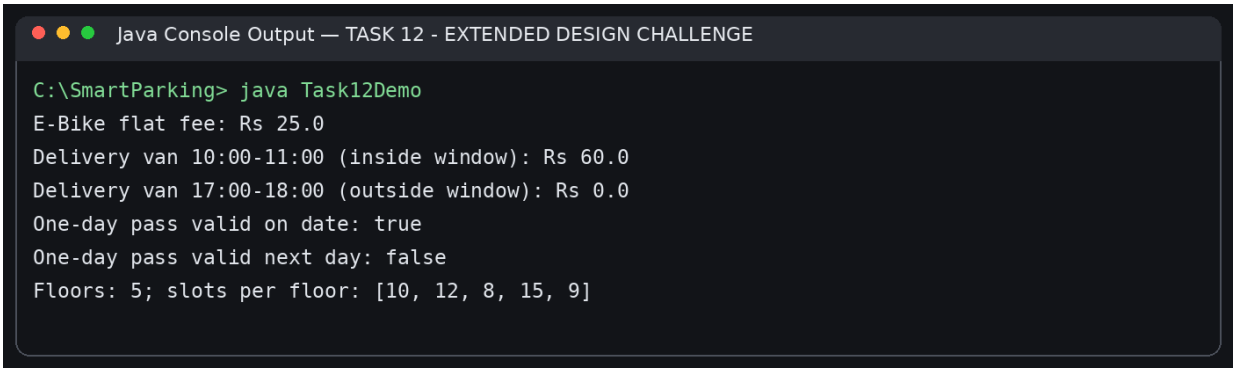
Source file: FeeDemo.java

TASK 11 - TEST CASE OUTPUT



Source file: ValidationDemo.java

TASK 12 - EXTENDED DESIGN CHALLENGE



Source file: Task12Demo.java